

**DESIGN AND IMPLEMENTATION OF
A SECURE PHONE-AS-CAR-KEY
PLATFORM FOR CAR RENTAL
SERVICES**

SHANE POWER - 20102592

CONTENTS

Part 1 – Project Specification	4
Introduction	4
Functional Summary	5
Assumptions.....	6
Functional Requirements	7
System Context.....	7
Detailed Requirements.....	7
Data Requirements	9
System Design Diagrams	11
Functional Decompositions.....	11
Sequence Diagrams	13
Part 2 – Implementation and Evaluation	16
Methodology	16
V-Model overview.....	16
Android App Driver User: V-model implementation	16
Phase 1: Requirements Analysis	16
Phase 2: System Design	18
Phase 3: Module Design	20
HomeViewModel BLE integration	20
Phase 4: Development	21
Biometric Authentication.....	22
Tamper and Replay Protection	22
Test Coverage	22
Python VACU V-Model implementation	23
Phase 1: Requirements Analysis	23
Phase 2: System Design	23
Phase 3: Module Design	24
Phase 4: Development	25
Android App: Manager User V-Model implementation.....	25
Phase 1: Requirements Analysis	26
Phase 2: System Design	26
Phase 3: Module Design	27
Phase 4: Development	27
App Screenshots.....	28
Implementation	31

Android Car Key App.....	31
Authentication	31
Security Implementation	31
Cybersecurity Goals	31
Threat Model.....	32
CANoe Configuration	35
Simulating the Janix Module.....	35
Manual Ignition Control	35
Cloud Monitoring Dashboard	36
Layout	36
Testing.....	37
Unit Testing.....	38
Integration Testing.....	38
System Testing.....	39
Security Testing.....	39
Acceptance Testing.....	40
Future Work / Limitations.....	40
Reflection	41
Conclusion	41
References	42

INTRODUCTION

The automotive industry is moving towards digital key systems that allow smartphones to replace traditional physical keys. Standards such as the Car Connectivity Consortium (CCC) Digital Key specification (Car Connectivity Consortium, 2026) cover the hardware level side of this, defining how phones, vehicles and key management servers interact across major vehicle manufacturers. Recent work has demonstrated practical man in the middle attacks against production smartphone based digital key systems, including a successful BLE attack on the Tesla Model 3 Phone Key (Xie et al., 2023), showing the threat for these systems is real world. However, fleet and rental operators have additional challenges on top of this. They need to assign, revoke, and monitor keys remotely across many vehicles, often without being able to rely on specialised hardware on the user's device. This project explores how a software prototype can address that issue while still applying the security techniques that production digital key systems rely on.

The project aims to design and implement a secure smartphone-based digital car key system, with a focus on car rental and car fleet management. The aim of the system is to replace traditional physical keys with securely provisioned digital keys. This will improve user convenience and reduce the administrative overhead and the security risks associated with handling physical keys. This is inspired by the shift in the car industry towards keyless vehicle access, and the growth of contactless, app-based car rental services.

The proposed prototype consists of a mobile Car Key App, a Vehicle Access Control Unit (VACU) running on a PC with Python, a cloud based management platform with an application for fleet managers, and a CAN interface using Vector CANoe. The Car Key App authenticates the user using biometric or PIN verification and initiates a secure unlock request. This request is transmitted via Bluetooth Low Energy (BLE) to the VACU, which performs access validation and cybersecurity checks like a vehicle lock ECU.

When the VACU receives a lock/unlock request, it verifies the command's signature against the user's registered public key from the cloud database and uses cybersecurity techniques such as checking the timestamp and cryptographic nonce. When access is approved, the VACU forwards the request to CANoe through a COM interface, which generates and transmits a real Controller Area Network (CAN) message to a Jaguar Land Rover vehicle testing rig. This message is used to wake the rig and simulate vehicle access behaviour, such as door lock and unlock events, showing interaction with real vehicle equipment.

The cloud backend provides fleet management functionality through a Manager App. This includes secure user authentication, digital key provisioning and revocation, temporary guest access management, and event logging. Fleet managers can remotely assign vehicles, monitor access activity, and revoke digital keys, allowing them to manage their fleet through a single app.

A comprehensive threat model will be developed to analyse potential cybersecurity risks in the system, such as replay attacks, relay attacks, and device compromise. These threats will be evaluated through controlled testing, and techniques to protect against these demonstrated such as authentication, nonces, timestamps, and secure communication channels between the system components. This will evaluate the security, practicality, and realism of a digital car key system from a cybersecurity point of view.

FUNCTIONAL SUMMARY

This project will develop a secure smartphone based digital car key system, aimed at improving user experience and fleet management for car rental services. The system replaces physical keys with a digital authentication and unlock process through a mobile app, which provides improved security and convenience for customers and fleet companies.

The Car Key App is the user's interface for authentication and vehicle access. Users unlock the app using biometrics(Face ID, fingerprint) or PIN and can then initiate an unlock or lock request. The app validates the user's session and encryption key against data from the cloud backend.

Once the request is validated, the app sends an unlock command to the Vehicle Access Control Unit (VACU) running on a PC, via Bluetooth Low Energy (BLE) . (Bluetooth Core Specification, 2024). The VACU processes the unlock request and enforces defined cybersecurity and authorisation rules, such as verifying ECDSA signatures against the user's registered public key, checking timestamp freshness and nonce uniqueness, similar to a real vehicle ECU. Rather than directly controlling the vehicle hardware, the VACU acts as the secure control point between the mobile app and the vehicle test environment.

Upon authorisation, the VACU forwards the request to CANoe, which is responsible for generating and transmitting a real Controller Area Network (CAN) message to the Jaguar Land Rover testing rig. This CAN message is used to wake the rig and trigger simulated vehicle access behaviour, such as door lock or unlock events.

The cloud backend manages user authentication, digital key provisioning, and event logging. Through the Manager App, fleet operators can create, assign, and revoke digital keys, manage temporary guest access, and monitor vehicle access activity. The cloud service supports scalable fleet operation and implements key revocation in the case of a lost or stolen device, as well as real-time updates of any access requests.

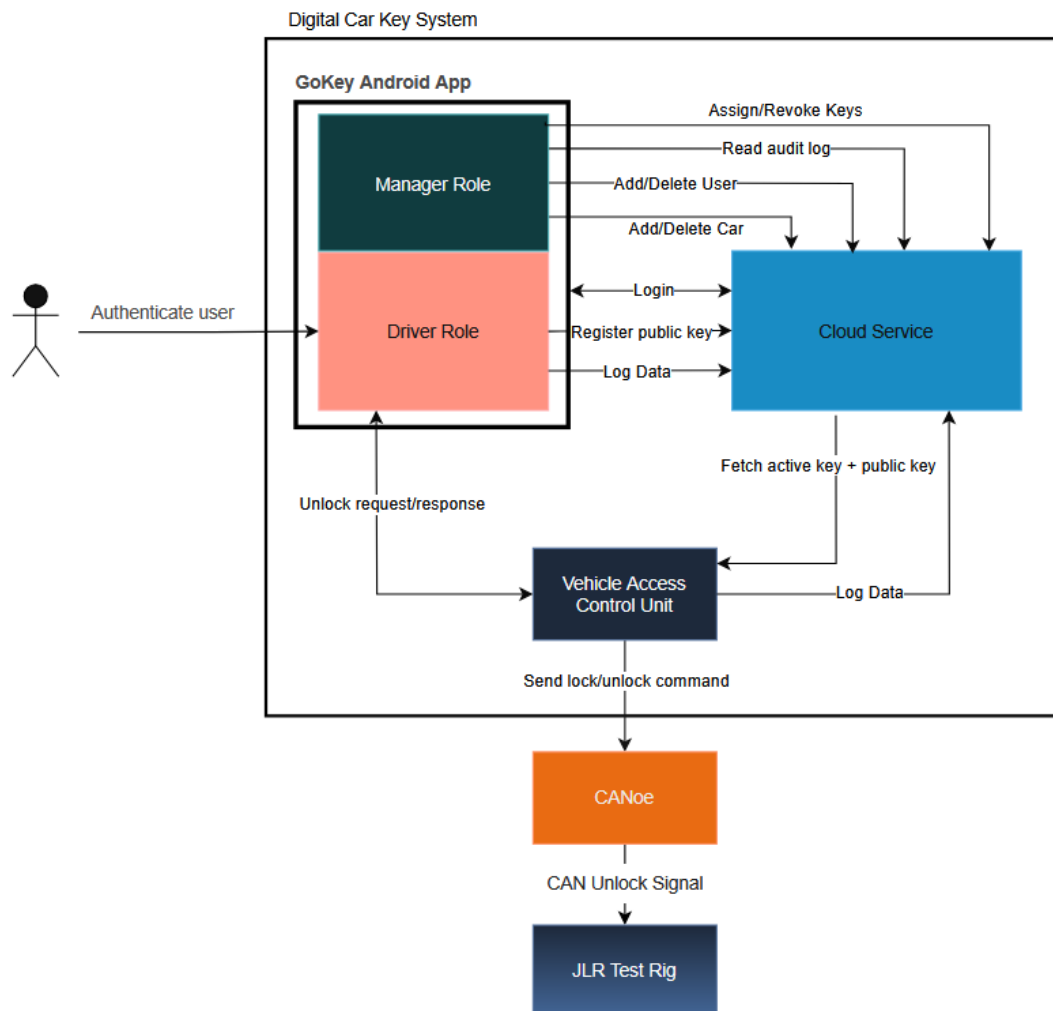
Overall, the system integrates mobile biometric authentication, Bluetooth Low Energy communication, CAN based vehicle communication, and cloud based fleet management to demonstrate a secure, contactless car key solution for modern rental and fleet use cases.

ASSUMPTIONS

- 1 The smartphone used for testing supports Bluetooth Low Energy (BLE) and can hold a stable BLE connection with the Vehicle Access Control Unit (VACU).
- 2 The Jaguar Land Rover (JLR) testing rig is available and operational, allowing CANoe to transmit real Controller Area Network (CAN) messages to wake the rig and trigger simulated vehicle functions such as door lock and unlock behaviour.
- 3 The cloud backend is available and provides reliable services for user authentication, digital key provisioning, and event logging throughout the testing period.
- 4 The Car Key App and the Manager App need to have access to a stable internet connection to communicate with the cloud backend.
- 5 The user's device should be able to implement secure key storage through the Android Keystore (Google, 2026) or an equivalent built-in security system.
- 6 All digital certificates and cryptographic key pairs required for authentication and secure communication can be generated securely and stored appropriately
- 7 The Vehicle Access Control Unit (VACU) is running on a system that supports BLE communication and has enough processing power to process and send unlock requests in a reasonable timeframe
- 8 Cloud service latency remains within acceptable limits to support near real time event logging and fleet management updates.
- 9 The prototype will be evaluated within a controlled hardware and network environment, and doesn't account for large-scale deployment or any hostile network conditions beyond those considered in the threat model.

FUNCTIONAL REQUIREMENTS

SYSTEM CONTEXT



DETAILED REQUIREMENTS

1. CAR KEY APP

1.1. User Authentication

- 1.1.1. The mobile app must allow users to authenticate using biometrics (fingerprint, Face ID) or a secure PIN.
- 1.1.2. Only authenticated users can access key management or vehicle unlock functions.
- 1.1.3. Authentication tokens must be stored securely within the phone's keystore.

1.2. Key Provisioning and Management

- 1.2.1 The app must synchronise with the cloud backend to validate key status, permissions, and user roles before issuing unlock commands.
- 1.2.2 The app must handle temporary guest keys with time-limited access or restricted permissions.
- 1.2.3 The app must reject commands if the key is revoked or expired.

1.3. Secure Communication and Unlock Process

- 1.3.1. The app must communicate with the BLE–CANoe interface via Bluetooth Low Energy (BLE).
- 1.3.2. Unlock commands must be bound to an authenticated user session and a valid KeyID.
- 1.3.3. The app must provide clear feedback to the user for command success, failure, or connection errors.

1.4. Data Handling

- 1.4.1 User's signing private key must be stored in the Android Keystore.
- 1.4.2 KeyID attributes include: KeyID, VehicleID, UserID, Expiry Date/Time, Permissions, Key status (active or revoked).
- 1.4.3 Unlock events generated by the app must be logged to the cloud with timestamp, user ID, vehicle ID, and action.

2. VEHICLE ACCESS CONTROL UNIT (VACU)

2.1 Communication Role

- 2.1.1. The VACU must establish and maintain BLE connections with authorised mobile devices.
- 2.1.2. The VACU must receive unlock and lock commands from the Car Key App in a reliable and timely manner.
- 2.1.3. The VACU must handle BLE disconnections and reconnections properly without system failure.

2.2 Access Control and Cybersecurity

- 2.2.1. The VACU must validate any incoming commands against the defined security and authorisation rules before forwarding the command to CANoe.
- 2.2.2. Commands must be rejected if the required validation checks fail.
- 2.2.3. The VACU must act as the only access point between the mobile application and the vehicle network.

2.3 Integration with CANoe

- 2.3.1. Upon successful validation, the VACU must forward authorised commands to CANoe.
- 2.3.2. The VACU must not directly generate or transmit CAN messages onto the vehicle network.
- 2.3.3. Communication between the VACU and CANoe must be reliable and have minimal latency.

3. CANOE AND VEHICLE NETWORK

3.1 CAN Message Generation

- 3.1.1 CANoe must generate valid CAN messages corresponding to actions on the test rig such as ignition status, door lock status.
- 3.1.2 CAN messages must conform to the expected format of the JLR testing rig.
- 3.1.3 CANoe must transmit messages reliably onto the CAN bus.

3.2 Test Rig Interaction

- 3.2.1 CANoe must be capable of waking the Jaguar Land Rover testing rig using CAN signals.
- 3.2.2 Simulated vehicle behaviour, such as door lock and unlock events, must be triggered by the CAN messages.
- 3.2.3 Monitoring of CAN messages sent and received to the test rig should be possible through CANoe.

4. CLOUD BACKEND AND MANAGER APP

4.1 User and Key Management

- 4.1.1 The cloud backend must manage all user authentication and role-based access control.
- 4.1.2 Fleet managers must be able to create, assign and revoke digital keys through the Manager App.
- 4.1.3 Changes to key status must be synchronised across the system in quick time.

4.2 Event Logging and Requirements

- 4.2.1 All unlock and lock attempts must be logged, including successful and failed actions.
- 4.2.2 Log entries must include timestamp, user ID, vehicle ID, action performed, and the result.
- 4.2.3 Fleet managers must be able to view access logs through the Manager App.

4.3 Security and Data Protection

- 4.3.1 Communication between components in the system and the cloud backend must use encrypted channels.
- 4.3.2 Sensitive data stored in the cloud must be protected using encryption and access controls.
- 4.3.3 Key revocation should be possible, in the case of a stolen or lost phone, or compromise of the vehicle.

1. User Data

- 1.1.** Each user will have a unique User ID which will be managed by cloud authentication.
- 1.2.** User profiles will include basic details such as name, email, public key of their generated signing key, and the assigned role (driver, manager)
- 1.3.** Authentication data such as passwords or biometrics will be securely stored and encrypted by secure cloud storage and the keystore on device.

2. Digital Key Data

- 2.1.** Each key will be represented by a KeyID which will be linked to a specific vehicle and user.
- 2.2.** The private key associated with each digital key will be stored on the phone keystore only.
- 2.3.** Attributes for KeyID include:
 - 2.3.1.** KeyID
 - 2.3.2.** VehicleID
 - 2.3.3.** UserID
 - 2.3.4.** Expiry Date and Time
 - 2.3.5.** Permissions
 - 2.3.6.** Key status (active or revoked)

3. Vehicle Data

- 3.1** Each vehicle entry in the database will have:
 - 3.1.1** VehicleID
 - 3.1.2** Vehicle Type / Model
 - 3.1.3** Associated Vehicle Access Control Unit
 - 3.1.4** Current Status
 - 3.1.5** Assigned User

4. Unlock Event Log

- 4.1** Every unlock attempt will be logged to the cloud
- 4.2** Each log entry includes
 - 4.2.1** LogID
 - 4.2.2** Timestamp
 - 4.2.3** UserID
 - 4.2.4** VehicleID
 - 4.2.5** Action (Unlock or Lock)
 - 4.2.6** Result (Success or Failure)
 - 4.2.7** Nonce

5. Vehicle Access Control Unit Data

- 5.1** The VACU will store:
 - 5.1.1** Unique VACU Identifier
 - 5.1.2** Cached public keys of valid users
 - 5.1.3** Latest valid nonces and timestamps for replay attack prevention
 - 5.1.4** Log data

6. CAN Bus data

- 6.1** CAN bus will send CAN messages to JLR rig to request unlock/lock

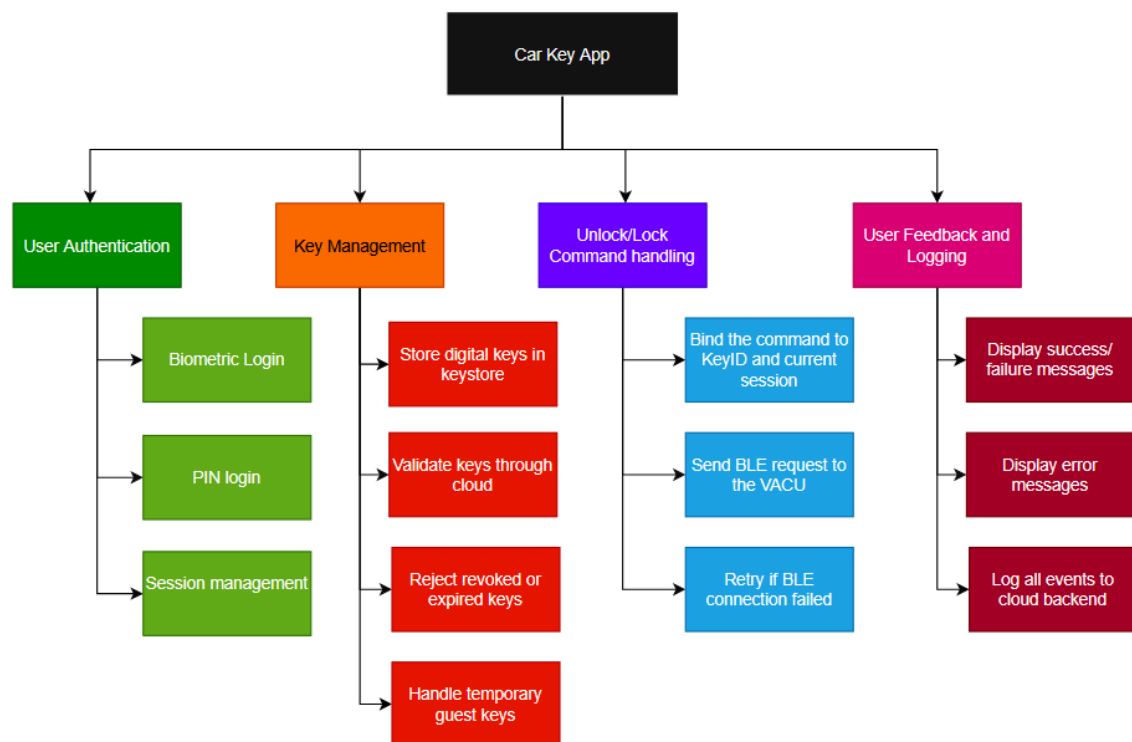
7. Data Security

- 7.1** All data transmitted between system components must be encrypted using secure communication protocols.
- 7.2** Secure cloud storage will handle secured storage of user, vehicle and event data
- 7.3** Keys and credentials will be stored using phone keystore and secure cloud storage.

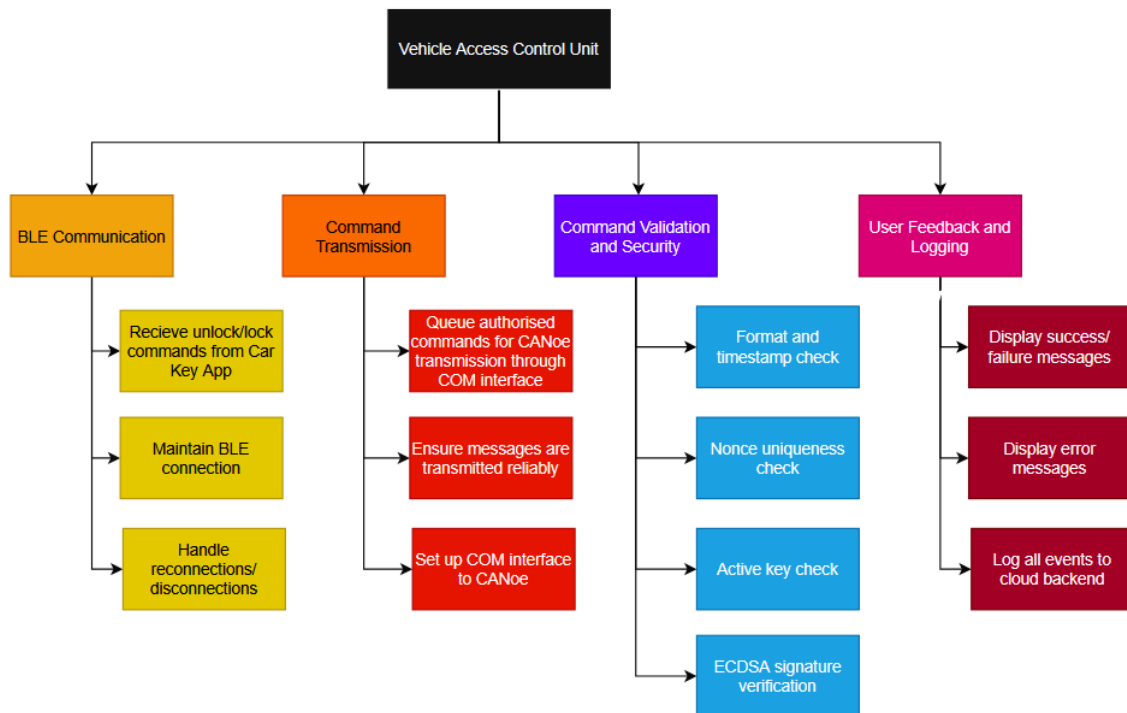
SYSTEM DESIGN DIAGRAMS

FUNCTIONAL DECOMPOSITIONS

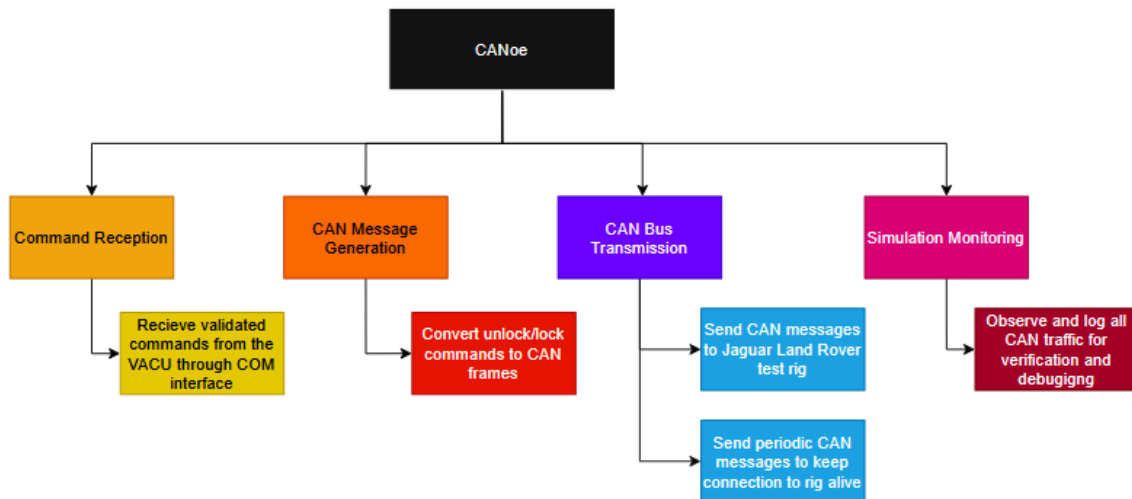
CAR KEY APP



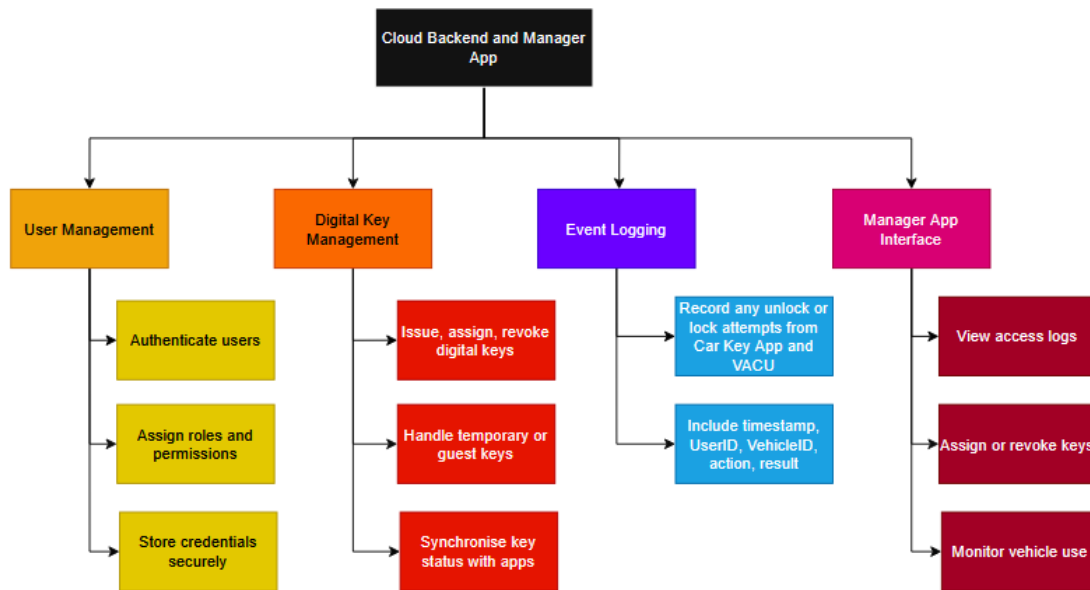
VEHICLE ACCESS CONTROL UNIT



CANOE

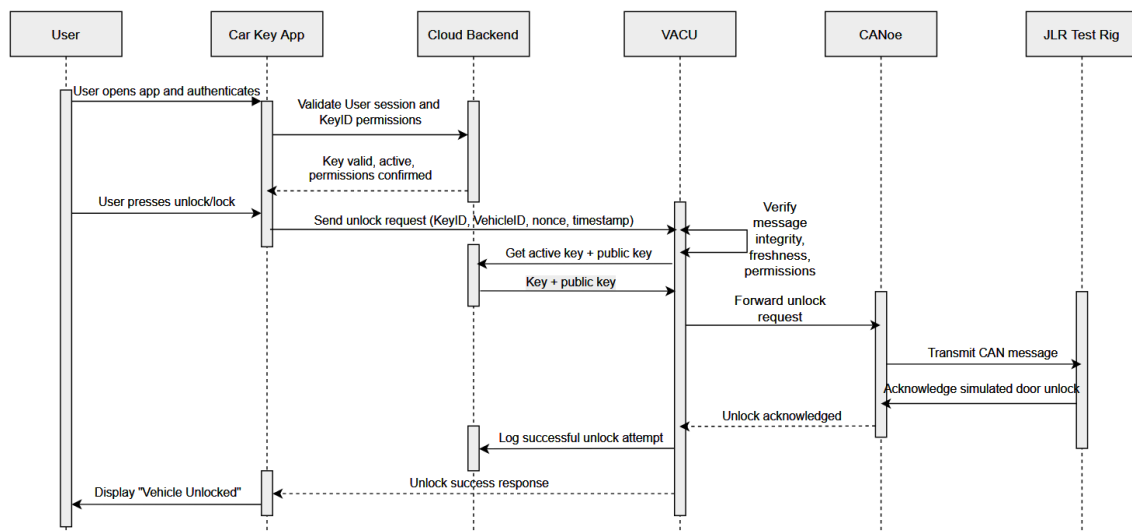


CLOUD BACKEND AND MANAGER APP

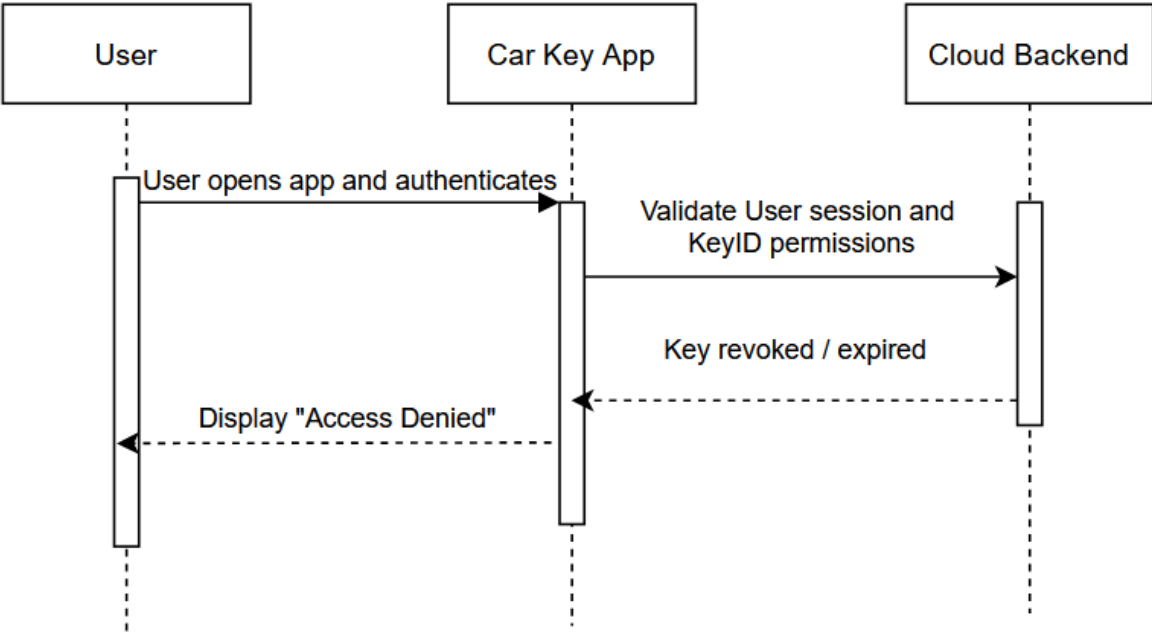


SEQUENCE DIAGRAMS

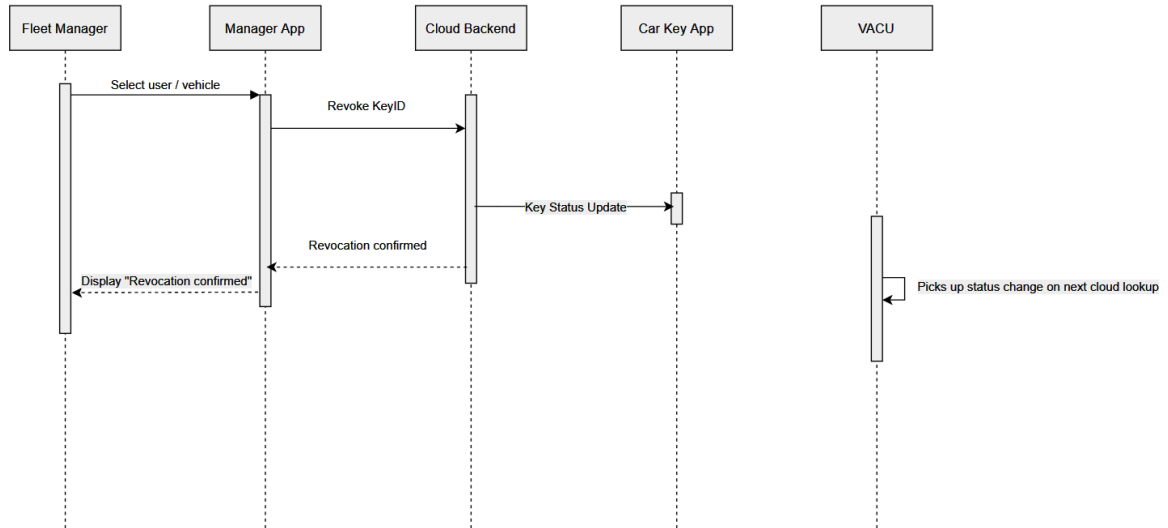
SUCCESSFUL UNLOCK/LOCK



UNLOCK FAILED (KEY INVALID)



MANAGER REVOKING KEY



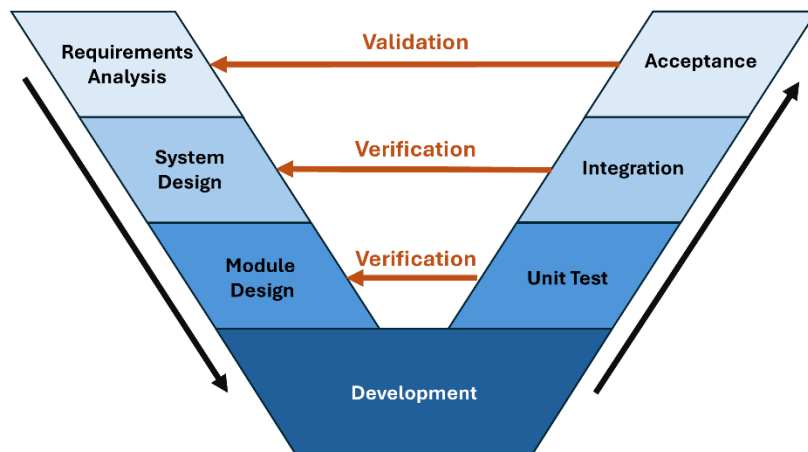
PART 2 – IMPLEMENTATION AND EVALUATION

METHODOLOGY

The V-Model (Forsberg and Mooz, 1991) was selected because it is the industry standard for safety-critical and automotive software - each development phase has a directly corresponding test phase, ensuring all requirements are verifiable and traceable throughout the project.

V-MODEL OVERVIEW

The V-Model structures development as a descending left arm (design/specification) and an ascending right arm (verification/testing), meeting at implementation at the bottom:



ANDROID APP DRIVER USER: V-MODEL IMPLEMENTATION

PHASE 1: REQUIREMENTS ANALYSIS

This phase directly corresponds with the Acceptance test phase. The goal of this phase is to define what the system must do, what data it must manage, and what the acceptance criteria are.

FUNCTIONAL REQUIREMENTS

ID	Requirement
FR-01	The app shall allow a registered user to log in using their email address and password via Firebase Authentication.
FR-02	The app shall display the vehicle and digital key assigned to the logged-in user, retrieved from Cloud Firestore.
FR-03	The app shall allow a user with an active digital key to send a lock command to the VACU over Bluetooth Low Energy.

FR-04	The app shall allow a user with an active digital key to send an unlock command to the VACU over Bluetooth Low Energy.
FR-05	The app shall prevent lock/unlock actions if the user's digital key has expired or been revoked.
FR-06	The app shall include a nonce and timestamp in every BLE command to prevent replay attacks.
FR-07	The app shall log every lock/unlock event (timestamp, userId, vehicleId, action, result, nonce) to Cloud Firestore. (Google, 2026)
FR-08	The app shall allow the user to log out, clearing the active session.
FR-09	The app shall display the current BLE connection status to the user.
FR-10	The app shall support temporary guest keys with an expiry date.

NON-FUNCTIONAL REQUIREMENTS

ID	Requirement
NFR-01	BLE command round-trip should complete within 3 seconds under normal conditions.
NFR-02	The app shall remain responsive during Firestore operations
NFR-03	The system shall log all events within 5 seconds of the action
NFR-04	Key revocation shall take effect on the next command attempt
NFR-05	The VACU shall reject payloads older than 30 seconds

DATA REQUIREMENTS AND DATA MODELS

The following data models were defined based on the functional requirements. These are implemented as Kotlin data classes and correspond directly to Firestore collections.

User

Represents a registered user of the system. Stored in the users Firestore collection.

```
data class User(
    val userId: String = "",
    val name: String = "",
    val email: String = "",
    val role: UserRole = UserRole.DRIVER
)

enum class UserRole {
    DRIVER, // Can only lock/unlock their assigned vehicle
    MANAGER // Can provision, assign and revoke keys
}
```

DigitalKey

Represents a provisioned digital key linking a user to a vehicle. Stored in the digitalKeys Firestore collection.

```
data class DigitalKey(
    val keyId: String = "",
    val vehicleId: String = "",
    val userId: String = "",
```

```

        val status: KeyStatus = KeyStatus.ACTIVE,
        val expiryDate: com.google.firebase.Timestamp? = null,
        val permissions: List<String> = emptyList()
    )

enum class KeyStatus {
    ACTIVE,        // Key is valid and can be used
    REVOKED,       // Key was explicitly revoked by a manager
    EXPIRED        // Key passed its expiry date
}

```

Vehicle

Represents a vehicle in the fleet. Stored in the vehicles Firestore collection.

```

3.3.4 data class Vehicle(
    val vehicleId: String = "",
    val make: String = "",
    val model: String = "",
    val registrationPlate: String = "",
    val currentKeyId: String? = null
)

```

VehicleEvent

Represents a logged lock or unlock event. Stored in the vehicleEvents Firestore collection. Every lock/unlock attempt is logged here for auditability.

```

data class VehicleEvent(
    val logId: String = "",
    val timestamp: com.google.firebase.Timestamp? = null,
    val userId: String = "",
    val vehicleId: String = "",
    val action: VehicleAction = VehicleAction.UNLOCK,
    val result: EventResult = EventResult.SUCCESS,
    val nonce: String = ""
)

enum class VehicleAction { UNLOCK, LOCK, UNKNOWN }
enum class EventResult { SUCCESS, FAILURE }

```

PHASE 2: SYSTEM DESIGN

This phase directly corresponds with the Integration test phase.

ARCHITECTURE DECISION

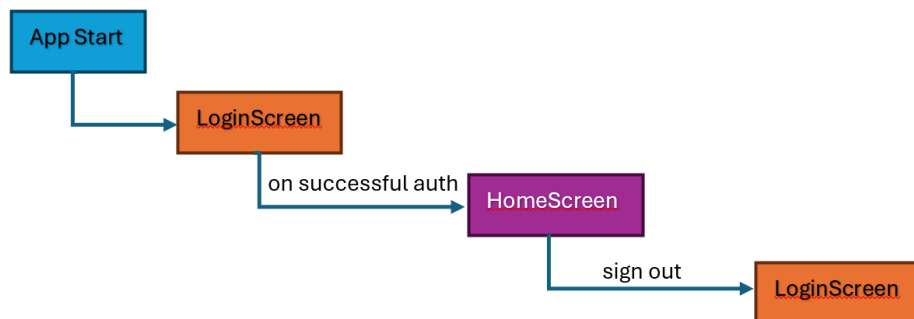
Adopted the MVVM (Model-View-ViewModel) architectural pattern (Google, 2026), which separates the UI logic (View) from business logic (ViewModel) and data (Model). This approach improves testability as Views and ViewModels can be tested in isolation. It also improves maintainability as code is more organised and easier to understand. The MVVM was implemented with the following layers:

Layer	Package	Responsibility
Data models	data/	Data classes that match Firestore schema

Auth	data/login/	Firestore Auth via AuthManager
Firestore stores	data/firestore/	One object per collection (KeyStore, VehicleStore, EventStore)
Repository	data/repository/	Single FirestoreRepository coordinating the stores
Service	service/	BleManager, Bluetooth logic implemented in phase 3
Viewmodels	ui/viewmodels/	AuthViewModel, HomeViewModel
Screens	ui/screens/	Composable screens
Components	ui/components/	Reusable Components for screens
Navigation	ui/navigation	AppDestinations, AppNavGraph
General UI	ui/general/	TopAppBarProvider, AppNavDrawer

NAVIGATION FLOW

There are two destinations defined in **AppDestinations.kt**. Login (route: "login") and Home (route: "home"). Login is the startDestination. On successful login, the Login route is popped from the back stack so the user cannot navigate back to it.



SYSTEM TEST SCENARIOS

ID	Scenario	Expected Result
ST-01	Launch app with no active session	LoginScreen is shown, HomeScreen is not accessible
ST-02	Login with valid Firebase credentials	Navigates to HomeScreen, Login removed from back stack
ST-03	Press back from HomeScreen after login	App exits, does not return to Login Screen

ST-04	Sign out from drawer	Returns to LoginScreen, clear session
ST-05	Launch app with existing active session	Goes directly to HomeScreen (auto-login)
ST-06	HomeScreen loads with active key	Vehicle name, reg and "Key Active" status displayed from Firestore
ST-07	HomeScreen loads with revoked key	Lock/Unlock buttons disabled, "Key Revoked" shown

PHASE 3: MODULE DESIGN

This phase focuses on breaking down the system architecture into individual modules. In this phase, each module's behaviour and responsibilities are defined along with the interfaces between them. This phase corresponds with the unit testing phase.

SECURITY MANAGER

This module constructs and parses the BLE payloads with the correct format. The payload has the format [command] : [nonce] : [timestamp]:[signature]. The nonce is a 32 character randomly generated hex string, and the timestamp uses the Unix millisecond value. The signature is produced by Android Keystore (Google, 2026) using ECDSA-SHA256 (NIST, 2023) over the first three fields. A private key is generated on the user's first sign-in, under a per-user alias. The corresponding public key is registered to the user's Firestore (Google, 2026) document, allowing the VACU to verify signatures without holding private key material. This format of message adheres to functional requirement FR-06 which aims to prevent replay attacks by using a unique nonce and timestamp, and a signature only the user's device can produce.

BLEMANAGER

This module implements the Bluetooth Low Energy GATT functionality. (Bluetooth Core Specification, 2024) The app advertises a custom service with a unique UUID. The app then accepts incoming connections from the VACU, which functions as the GATT central. The app notifies the VACU upon connection using a GATT characteristic. The BleManager module only starts once the user grants Bluetooth permissions for the app.

HOMEVIEWMODEL BLE INTEGRATION

The HomeViewModel was updated to integrate BLE status into the UI viewmodel, checking on unlock/lock for an active BLE connection and showing an error on the UI if commands are sent without being connected to the VACU.

TEST DESIGN

Unit tests were created for SecurityManager to check the payload structure, nonce generation and checks validity of payload throughout parsing process. Integration tests were created for BleManager to verify the Bluetooth service initialises correctly and starts as disabled.

PHASE 4: DEVELOPMENT

The development phase focused on completing the CarKey system, implementing cryptographic signing for unlock/lock commands, and adding biometric authentication.

ECDSA SIGNING DEVELOPMENT

Cryptographic signing of BLE payloads was implemented into the SecurityManager. The payload format was extended to four fields:

[command] : [nonce] : [timestamp] : [signature]

The signature is generated by Android Keystore using ECDSA-SHA256 (NIST, 2023) over the first three fields. The message is encoded using Base64 for transport. The nonce is a randomly generated 32-character hex string, with the timestamp using a Unix millisecond value. The payload is deemed valid only if its timestamp falls within a 30 second age window with a 10-second clock skew tolerance, which gives an effective validity range of 40 seconds.

When the user signs into the app for the first time, an ECDSA P-256 key pair is generated using Android Keystore. The key is stored under a unique alias based on the user ID. The private key is stored inside Android Keystore and cannot be extracted.

The public key is encoded and stored in the user's Firestore record. This lets the VACU verify any command sent by that user, without needing access to the user's private key.

When a command is received, the VACU retrieves the user's public key and uses it to verify the signature. Since the private key never leaves the device, neither the cloud backend nor the VACU can generate valid commands.

This is an improvement over the earlier version of the system that used a HMAC-SHA256 secret stored per-key in Firestore. This was a risk because storing a shared signing secret in the cloud could result in a

cloud breach exposing the signing key directly. Switching to an asymmetric approach means the only material in Firestore is the public key, which can't be used to produce commands.

BIOMETRIC AUTHENTICATION

Biometric authentication was implemented into the login flow using the AndroidX biometric library (Google, 2026). When the app is launched, the app checks for an active Firebase session. If the user is logged in, the user is prompted to use fingerprint or face scanning authentication to be granted access. If the device lacks biometrics, or if biometrics failed, the app will revert back to email/password login. This ensures that a stolen unlocked phone cannot issue unlock commands.

TAMPER AND REPLAY PROTECTION

The use of ECDSA signatures, along with the nonce cache and timestamp window, prevents three forms of attack:

- Replay Attack - nonce: each nonce is unique, so a previously generated payload can't be resent to trigger an action again. The VACU maintains a cache of nonces seen within the current replay window, in memory. A payload carrying a previously seen nonce is rejected even if the timestamp is still valid
- Replay attack – timestamp: payloads older than 30 seconds are rejected.
- Payload tampering: changing any part of the payload will invalidate the signature

TEST COVERAGE

Unit tests were created covering nonce generation and parsePayload payload rejection functionality. Tests for key signing and Keystore signing path are covered in SecurityManagerInstrumentedTest.

PYTHON VACU V-MODEL IMPLEMENTATION

PHASE 1: REQUIREMENTS ANALYSIS

This phase corresponds to the Acceptance test phase. The goal of this phase is to define what the VACU must do, the data it must manage, and the acceptance criteria.

FUNCTIONAL REQUIREMENTS

ID	Requirement
VACU-FR-01	The VACU shall establish and maintain a BLE connection with the Car Key App
VACU-FR-02	The VACU shall receive lock and unlock payloads from the Car Key App over BLE
VACU-FR-03	The VACU shall validate the format and timestamp of every payload received, rejecting any that fail
VACU-FR-04	The VACU shall maintain a nonce cache of seen nonces within the validity window, and reject any repeated nonce
VACU-FR-05	The VACU shall verify the key in the payload is active through querying Firestore, rejecting any revoked or expired keys
VACU-FR-06	The VACU shall verify the ECDSA-SHA256 signature of every payload, using the public key registered to the user assigned to the active key, and rejecting any payload that fails verification
VACU-FR-07	The VACU shall forward payloads to CANoe only if they pass all four stages of verification, and log every accepted or rejected payload to Firestore

NON-FUNCTIONAL REQUIREMENTS

ID	Requirement
VACU-NFR-01	Payload validation shall complete within 2 seconds under normal network conditions
VACU-NFR-02	The VACU shall handle BLE disconnections gracefully without crashing
VACU-NFR-03	The VACU will cache Firestore key lookups with a TTL, to prevent exhausting cloud quota
VACU-NFR-04	The VACU shall remain operational over multiple repeated lock and unlock requests without restart

PHASE 2: SYSTEM DESIGN

This phase directly corresponds to the integration test phase.

ARCHITECTURE DECISION

The VACU was implemented with Python using three core modules. The bleak library (Blidh, 2022) handles all BLE communication as the GATT central role. The firebase-admin SDK handles Firestore operations using a service account, which bypasses client-side security rules and operates with full read access. The pywin32 library implements the COM interface used to communicate with CANoe. This separation of responsibility means each module can be developed and tested independently.

Module	File	Responsibility
BLE Handler	bletocanoe.py	BLE connection, notification handling, dispatch to validation
Payload Parser	payload_parser.py	Payload validation, timestamp window check, nonce cache, ECDSA signature verification
Cloud Client	cloud.py	Firestore key lookup, event logging, TTL cache
Threat Simulator	threat_sim.py	Controlled attack simulation for testing

SYSTEM TEST SCENARIOS

ID	Scenario	Expected Result
VACU-ST-01	Receive a valid signed payload within the timestamp window	Payload accepted, command forwarded to CANoe, success event logged
VACU-ST-02	Receive a payload with an expired timestamp	Payload rejected at stage 1, failure event logged
VACU-ST-03	Receive a payload reusing a nonce already seen within time window	Payload rejected at stage 2, failure event logged
VACU-ST-04	Receive a payload with a tampered command field	Payload rejected at stage 4 with signature mismatch, failure event logged
VACU-ST-05	Receive a payload for a revoked key	Payload rejected at stage 3, failure event logged
VACU-ST-06	BLE client disconnects mid-session	VACU handles disconnection without crash, rescans and reconnects

PHASE 3: MODULE DESIGN

This phase breaks down the system into individual modules, and defines the interfaces between them.

PAYLOAD_PARSER.PY

This module is responsible for parsing and validating BLE payloads, enforcing timestamp freshness, maintaining the nonce cache and verifying ECDSA signatures. The **validate_payload** function checks the payload has four colon separated field, and that the timestamp is within the 30 second window. It also checks nonce uniqueness thorough the **is_unique_nonce** function, which holds recently seen nonces in

cache. The **verify_ecdsa_signature** function gets the user's X.509 public key with the cryptography library, and verifies the signature against the payload string.

CLOUD.PY

This module handles all interaction with Firestore. The **get_active_key** function returns the active digital key for the VACU's assigned vehicle. The results are cached for 10 seconds so the Firestore read quota is not exhausted from the constant BLE traffic. The **get_user_public_key** function returns the public key for the user who owns the active key, and caches it for 30 seconds. The **log_event** function writes a document to vehicleEvents in the Firestore, which stores the action, result, nonce, and failure reason if applicable. The VACU accesses Firestore through the Firebase Admin SDK using a service account.

PHASE 4: DEVELOPMENT

The development phase focuses on implementing the four stage validation process, integrating the Firestore cloud client and fixing BLE problems that were found during testing.

When a BLE payload is received, it passes through four sequential checks.

Stage 1: Validates the format, command, and timestamp window.

Stage 2: Enforces nonce uniqueness through checking nonce cache.

Stage 3: Checks the active key state in Firestore.

Stage 4: Verifies the ECDSA signature using the user's registered public key.

If any stage fails, the payload is rejected and logged to cloud with the failure reason.

The validation logic was originally implemented as a three-stage pipeline using HMAC-SHA256, with a per-key secret stored in Firestore. This approach proved to be flawed as it held the key secret in plaintext in Firestore, only relying on the Firestore Security Rules to secure it. The system was migrated to use ECDSA with hardware backed key generation using Android Keystore. A fourth stage using the nonce cache was added, to distinguish within-window replay attacks on the cloud dashboard,

During integration testing, it was found that the active key lookup was sending an excessive amount of read requests to Firestore and exhausting the quota, since it did a key lookup for every payload sent. A 10 second TTL cache was implemented into **get_active_key** and 30 second TTL cache to **get_user_public_key**, which dropped read counts significantly.

Two BLE issues were fixed during this phase. The first issue was a duplicate UUID error as the Android App was advertising the same characteristic UUID under two services. This was fixed by moving to using one GATT service. The second issue was the VACU freezing when the BLE connection dropped. This was fixed by using an outer loop that rescans and discovers BLE devices upon disconnect.

ANDROID APP: MANAGER USER V-MODEL IMPLEMENTATION

The manager app is not its own separate Android app, it's a separate part of the same GoKey app used by the drivers, enabled when the signed in user has the MANAGER role on their Firestore profile. When the Firebase Authentication is done, the user's role is read by **AuthManager**, and the navigation graph either brings the user to the driver home screen, or the manager dashboard. This approach keeps a single codebase, the same sign in behaviour and security model across both users.

PHASE 1: REQUIREMENTS ANALYSIS

This phase corresponds to the Acceptance test phase. The goal of this phase is to define what the Manager App must do, the data it must manage, and the acceptance criteria.

ID	Requirement
MGR-FR-01	The app shall route signed in users with MANAGER role to the Manager dashboard, and users with role DRIVER to the driver Home screen
MGR-FR-02	A manager shall be able to create a new user with a specified email, name and role
MGR-FR-03	A manager shall be able to add and remove vehicles from the fleet
MGR-FR-04	A manager shall be able to assign a digital key linking a user to a vehicle
MGR-FR-05	A manager shall be able to revoke an active digital key
MGR-FR-06	A manager shall be able to view the audit log of all lock and unlock events across the fleet
MGR-FR-07	A driver shall not be able to access any manager screen

ID	Requirement
MGR-NFR-01	Role based access shall be enforced both client side and server side
MGR-NFR-02	Key revocation shall take effect on the VACu within the key cache window (10 seconds)
MGR-NFR-03	The dashboard shall load fleet state in under 3 seconds under normal cloud conditions

PHASE 2: SYSTEM DESIGN

The manager screens reuse the same MVVM model as the rest of the app. The **ManagerViewModel** holds the state for users, vehicles, keys and events. It exposes the **assignKey**, **revokeKey**, **addUser**, and **addVehicle** functions. These functions call through **FirestoreRepository** to the **Firestore** stores already used in the app (**UserStore**, **VehicleStore**, **KeyStore**, **EventStore**)

The routing is handled at the navigation layer. After user sign in, **AuthManager** reads the role field of the user's profile, and the navigation graph branches to either Home or Manager as the destination. The login route is popped off the back stack so the user cannot navigate back to it through the bottom nav buttons.

ID	Scenario	Result
MGR-ST-01	Manager signs in with valid credentials	Manager dashboard is shown, driver home screen is not reachable

MGR-ST-02	Driver signs in with valid credentials	Driver home screen is shown, manager screen is not reachable
MGR-ST-03	Manager assigns a key to a driver, driver signs in and presses unlock	Test rig response, success event logged with user ID
MGR-ST-04	Manager revokes an active key	Next unlock attempt by driver is rejected and logged as key revoked, if unlock was triggered within cache window. Afterwards, attempt will reject due to lack of active key for user.
MGR-ST-05	Manager adds a new vehicle, then assigns a key for it	Vehicle appears in the fleet list, key assignment success

PHASE 3: MODULE DESIGN

Module	File	Responsibility
Manager view model	ui/viewmodel/ManagerViewModel.kt	Holds combined state for users, keys, vehicles and events. Exposes assign, revoke, add functions
Manager screens	ui/manager/	Composable screens for user list, vehicle list, key assignment, audit log
Role based routing	ui/navigation/AppNavigation.kt	Reads user role after sign in, branches to driver or manager destinations
Repository	data/repository/FirebaseRepository	Single firebase coordinator, delegates to appropriate collection stores, reused by driver and manager view models.

PHASE 4: DEVELOPMENT

The development phase focused on role-based routing, the flow for key assignment, and Firestore Security Rules to prevent unauthorised cloud access.

When authentication is complete, **AuthManager** reads the user's profile and resolves their **UserRole**. The login flow then routes the user to the correct destination through **NavController.navigate()**. The login route is removed from the back stack to prevent navigating back to the login screen through the on screen navigation.

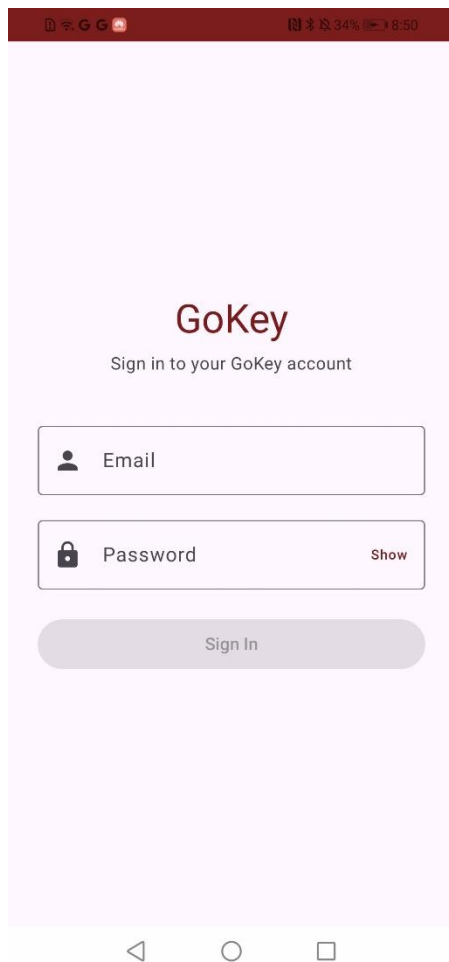
ManagerViewModel.assignKey creates a **DigitalKey** with a unique UUID, the user and vehicle IDs, the key's ACTIVE status and an expiry timestamp. This is written to cloud using **KeyStore.assignKey**. When a manager assigns a key, there is no cryptographic material being dealt with. The signing keys are

generated on the user's device when they first sign in, and the public key is registered to their user document automatically. The manager never has to handle cryptographic keys.

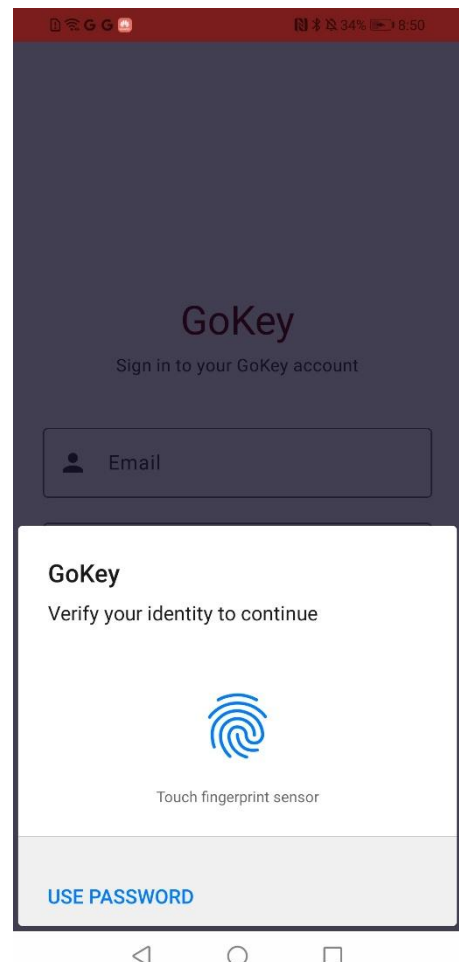
Key revocation is done through a Firestore update, setting the key's status to REVOKED.

Firestore Security Rules are implemented for manager functions. Only managers can write to the **digitalKeys**, vehicles and user collections for other users. Drivers can only read their own profile and digital key and only update the **publicKey** field on their own record. The audit log is write only from the VACU's service account and read only for managers.

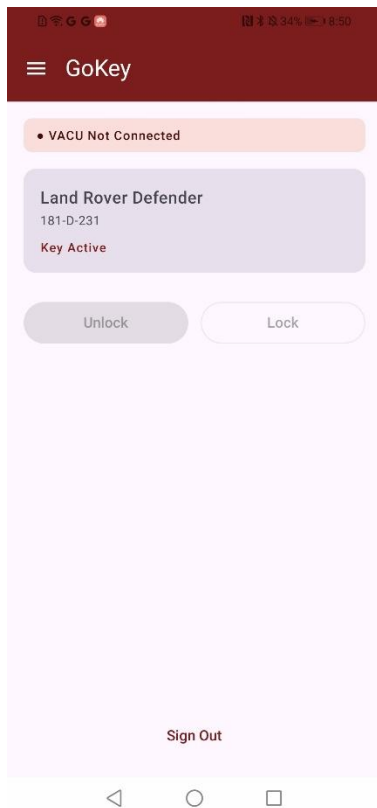
APP SCREENSHOTS



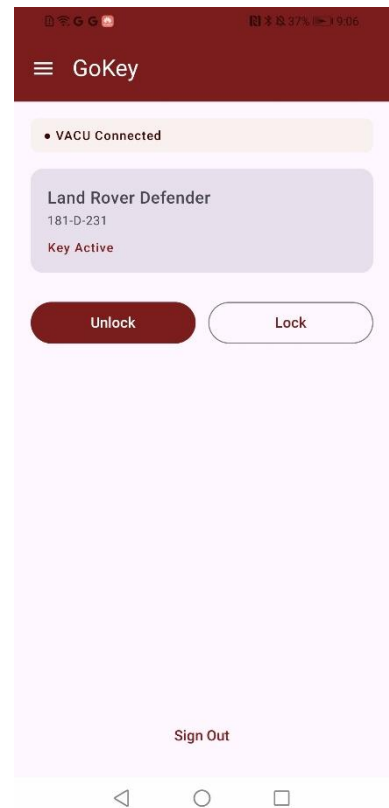
1: Login screen



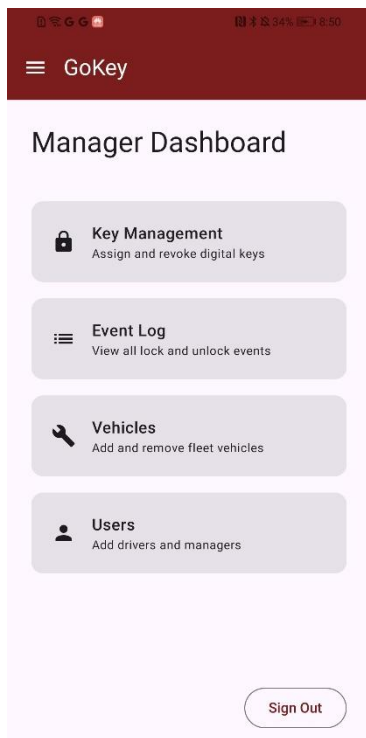
2: Biometric screen



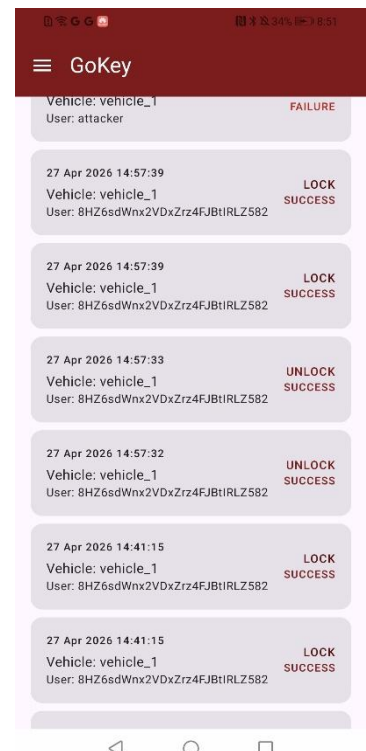
3: Driver screen, VACU disconnected



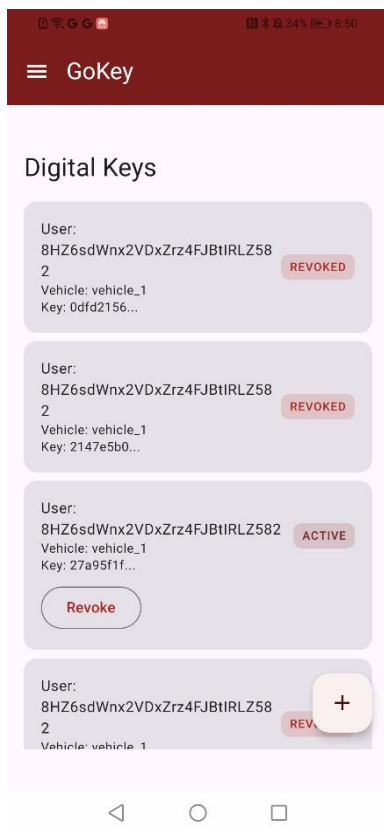
4: Driver screen, VACU connected



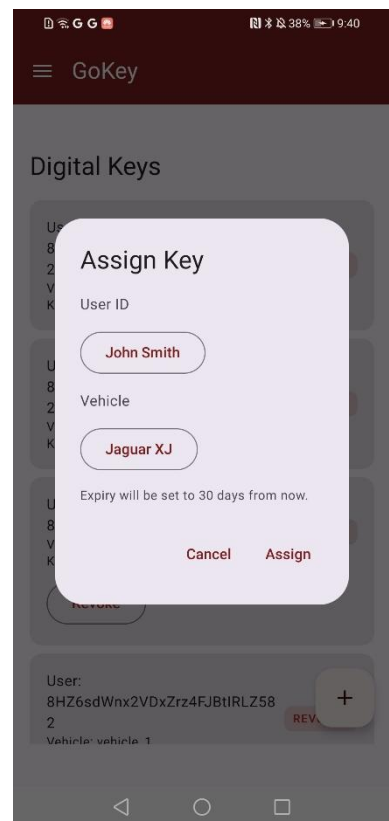
5: Manager dashboard



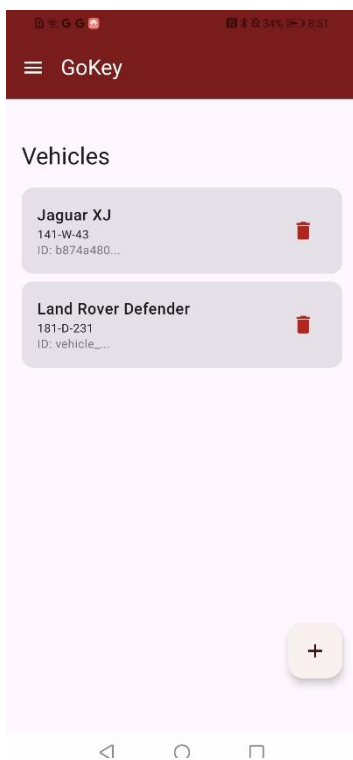
6: Event log



7: Digital Keys screen



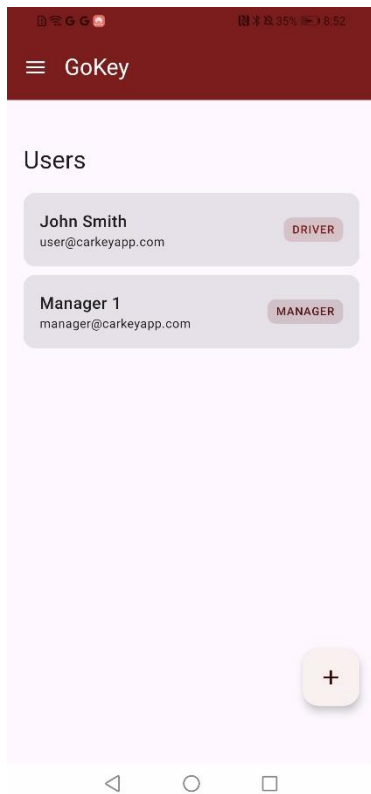
8: Assign Key Screen



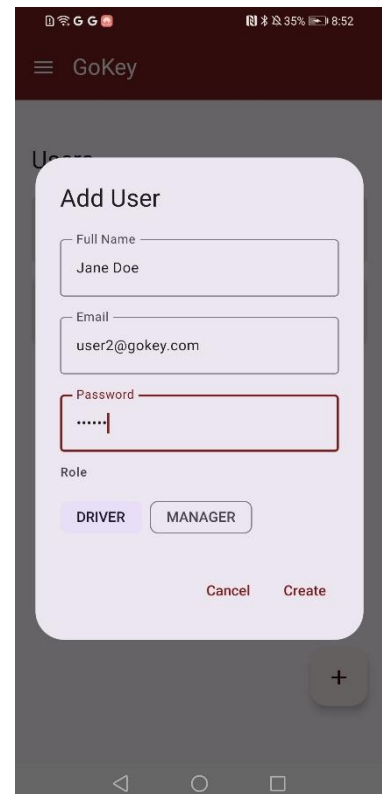
9: Vehicles screen



10: Add Vehicle Screen



9: Users Screen



10: Add Users screen

IMPLEMENTATION

The system was implemented across four components: the Android Car Key App, the Python Vehicle Access Control Unit (VACU), the Firebase cloud backend and Manager App, and a Streamlit monitoring dashboard. Each component was developed following the V-Model process documented in the previous section.

ANDROID CAR KEY APP

The Car Key App was built using Android Studio with Kotlin, and Jetpack Compose for the UI (Google, 2026). The application follows the MVVM architecture which separates the UI logic from business logic and data.

AUTHENTICATION

Authentication of users is handled by Firebase Authentication, through email and password, and Google Authentication. On login AuthManager signs the user in, and finds their user from Firestore to resolve their user role. The role will determine what part of the app they're brought to. Drivers are brought to the vehicle home screen, and managers brought to the management dashboard.

SECURITY IMPLEMENTATION

CYBERSECURITY GOALS

These goals aim to align with the OWASP Mobile Application Security Verification Standard (MASVS), which defines verification requirements for mobile app security across authentication, cryptography, network communication and resiliency. (OWASP, n.d)

1. Secure Authentication

- 1.1. Ensure that only authorised users can unlock vehicles.
- 1.2. Authentication should be implemented through biometrics or secure PIN.
- 1.3. In the case of a phone being stolen or lost, unauthorised use should be prevented through key revocation.

2. Data Confidentiality

- 2.1. All sensitive information (nonces, KeyIDs, cryptographic keys) should be protected.
- 2.2. Any Bluetooth Low Energy communication should be protected by ensuring BLE Secure Connections are used, by implementing pairing and bonding.
- 2.3. Communications between the Car Key App, VACU and the Cloud Backend should be protected with TLS.

3. Data Integrity

- 3.1. It should be ensured all unlock requests are unaltered and genuine.
- 3.2. Replay attacks should be prevented by implementing challenge response techniques, such as nonces, signatures, and timestamps.
- 3.3. The VACU should validate every request before sending any signals to the vehicle network.

4. Data Availability

- 4.1. In the case of a network failure, the system should handle the failure gracefully and return to a safe resting state.

5. Accountability

- 5.1. There should be a secure and complete log of all unlock or lock events.
- 5.2. The logs should include the timestamp, UserID, VehicleID, the action, the result, and nonce.

THREAT MODEL

These threat sources and attack types are framed against the OWASP Mobile Application Security Testing Guide (MASTG), which provides structured information and methods for identifying mobile app security weaknesses. (OWASP, n.d)

POTENTIAL THREAT SOURCES:

- **External Attacker:** An attacker trying to access a vehicle without a valid provisioned key
 - **Insider Threat:** A manager with privileges, abusing their authority to assign unauthorised keys
 - **Man In the Middle:** An attacker intercepting Bluetooth or cloud communication
 - **Replay Attacker:** An attacker who captures a successful unlock command and replays it to trigger an unlock.
 - **Compromised Device:** A connected phone or PC that has been tampered with
-

POTENTIAL ATTACK TYPES:

1. Unauthorised Access from External Attacker

In this attack, the attacker has obtained a KeyID or valid credentials from a phone or the cloud. This would allow the attacker to unlock a vehicle.

To mitigate this:

- Biometric authentication implemented and required upon launching the app when an active session is detected, which prevents use of a stolen unlocked phone

- Every BLE payload carries an ECDSA-SHA256 signature, which is generated using the user's private key in Android Keystore. The private key cannot be exported, so a stolen phone or compromised cloud cannot be used to produce valid signatures.
- Key revocation ensures compromised keys are deactivated immediately through the Manager App. Any key that does not have ACTIVE status in Firestore is rejected by the VACU.
-

2. **Replay Attack**

Replay attacks against keyless entry systems have been studied since the introduction of RF keyfobs (Alrabady and Mahmud, 2005) The same principles apply for a BLE based smartphone key. In this attack, the attacker captures a valid unlock or lock payload sent through BLE from a user's phone to the VACU. The attacker can then later transmit this payload to the VACU again to trigger a successful unlock.

To mitigate this:

- Nonces are implemented for each unlock request. A nonce is short for "number once" and is a randomly generated number that is used only once in a communication. For this implementation, the nonce is a randomly generated 32 character hex string. This prevents two payloads from being identical. The VACU stores a cache of nonces seen within the current replay window. Any payload that is carrying a nonce that has already been processed is rejected, even if the timestamp is valid. This prevents an attacker from capturing and retransmitting a payload before the timestamp window closes
- A payload includes a Unix millisecond timestamp. This allows the VACU to reject any payload with a timestamp older than 30 seconds, stopping attackers from replaying a payload at a later time.
- The payload's format is [command]:[nonce]:[timestamp]:[ecdsa-signature]. The ECDSA signature covers the three previous fields, stopping an attacker from modifying the timestamp as the signature will become invalid.

3. **Man In the Middle Attack**

In this attack, an attacker intercepts the BLE transmission between the phone and the VACU and tries to change the payload, for example changing a lock command to an unlock command and then sending the payload to the VACU. A man in the middle attack was demonstrated against a Tesla Model 3 phone key through BLE, evidence that this attack has already been proved against production technology. (Xie et al., 2023)

To mitigate this:

- Each payload is signed with ECDSA-SHA256 before it is sent. A four-stage validation process is done by the VACU on every payload received. It firstly validates the payload's format and checks that the timestamp is within the 30 second window. Secondly, it a temporary in-memory cache stores recently seen nonces. If a nonce is reused, the message is rejected, preventing replay attack within the valid time window. Thirdly, it queries Firestore and checks if the key is ACTIVE for the target vehicle. Finally, it verifies the ECDSA signature using the user's public key from Firestore and recomputes the expected signature. If the computed signature does not match the received signature, the payload is then rejected.
- Modifying any part of the payload causes the verification to fail at the signature verification stage. This was tested through controlled tamper testing, where a valid lock payload had been modified and the command field had been changed to unlock. The VACU rejected the payload with a signature mismatch error.

4. **Cloud Data Breach**

In this attack, the cloud backend is compromised. Private user data could be access, and access to key provisioning options could be accessed by an attacker, allowing unauthorised devices to be allocated keys.

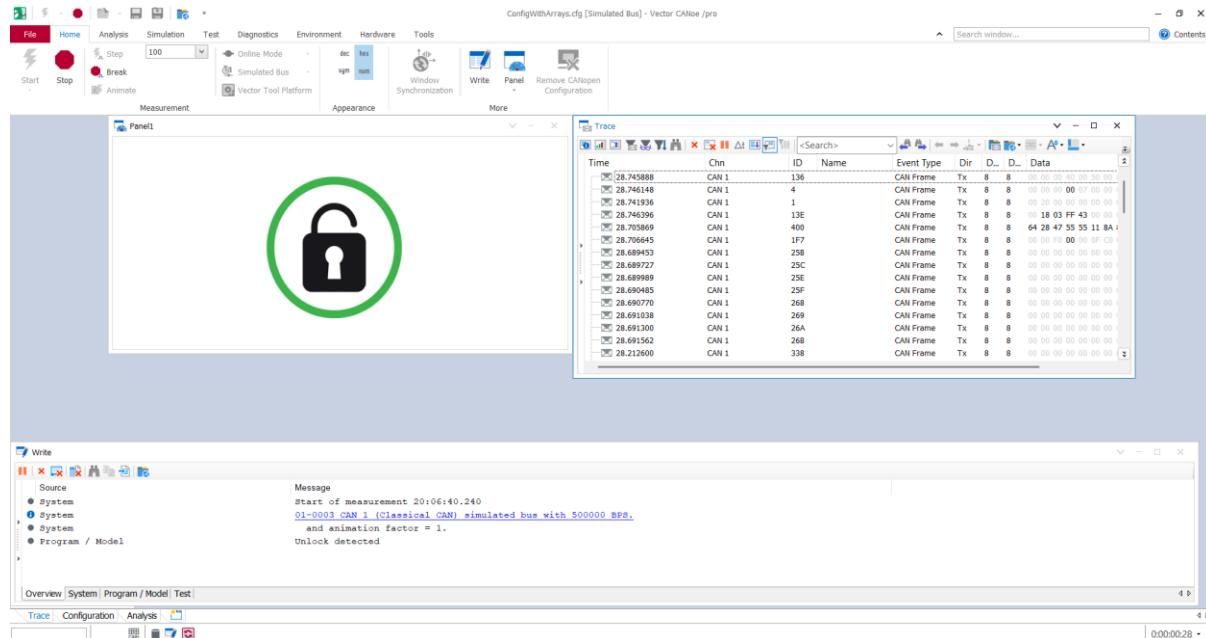
To mitigate this:

- Firestore Security Rules are implemented which restrict access to documents by user role. Drivers can only read their own key document and can only update the publicKey field on their own record. Managers can issue and revoke keys, but cannot impersonate users. Any unauthenticated access is blocked.
- The VACU accesses Firestore with the Firebase Admin SDK through a service account. This operates independently of client side authentication, and can't be impersonated by a compromised user's session.
- All communications between the VACU, app and Firebase is over TLS-encrypted channels.
- Managers can revoke all active keys in the event of the cloud being compromised, stopping secure keys being used any further.

The mitigation of these threats will be demonstrated and each attack vector will be tested in a controlled environment, to show the system's resilience against cybersecurity attack.

CANOE CONFIGURATION

Vector CANoe (Vector, 2023) was used as the interface between the VACU and the JLR test rig. It runs on the same PC as the VACU, and is controlled through a COM interface from **bletocanoe.py**. The CANoe configuration file is **ConfigWithArrays.cfg**.



SIMULATING THE JANIX MODULE

The test rig is normally controlled by a Janix module, a small controller box on the side of the rig. It transmits ignition status and other vehicle status messages, and has controls to set ignition state, allowing the rig to “wake up” from sleep mode. Without these simulated messages, the rig will not respond to commands sent to it. CANoe takes the place of the Janix module in this project.

The PC was first connected to the test rig with a Vector VN7610 CAN interface, which connects to the PC using USB and outputs to a CAN serial port, connected to the HMI CAN port on the rig. The Janix was set to Power Mode 7 (ignition on), and CANoe’s logging block function was used to capture the CAN frames transmitted by the Janix. The capture was then saved as a CANoe replay file.

In the project configuration, the captured CAN messages are played back in real time through a CAN replay block, which the test rig interprets as messages from the Janix module.

MANUAL IGNITION CONTROL

The ignition state CAN message is excluded from the replay block and generated manually by CAPL code in the configuration. The value it transmits is bound to a system variable.

Two power modes are used:

- Power Mode 0: ignition off, rig in sleep mode
- Power Mode 7: ignition on, rig is awake and accepting commands

```

variables
{
    message lockstatus msg; // Define the message to send
    msTimer lockTxTimer;
}

on sysvar_change sysvar::Vehicle::DoorLock {
    if (@this == 1){
        write("Lock detected");
        msg.byte(2) = 0x00;
    } else {
        write("Unlock detected");
        msg.byte(2) = 0x07;
    }

    setTimer(lockTxTimer, 0); // Start periodic sending
}

on timer lockTxTimer
{
    msg.ID = 0xFF;
    msg.DLC = 8;

    msg.byte(0) = 0x00;
    msg.byte(1) = 0x00;
    msg.byte(3) = 0x00;
    msg.byte(4) = 0x00;
    msg.byte(5) = 0xC0;
    msg.byte(6) = 0x00;
    msg.byte(7) = 0x00;

    output(msg);
    setTimer(lockTxTimer, 100); // Send every 100 ms
}

```

The system variable **Vehicle.DoorLock** is exposed to the COM interface. When an unlock command is accepted by the VACU, it flips the value of the **DoorLock** variable and the CAPL code block sends the corresponding ignition message. A CANoe panel is created to display the current **DoorLock** status.

CLOUD MONITORING DASHBOARD

The dashboard is a Streamlit (Streamlit.io, 2025) application running through Python that reads from Firestore using a service account. It shows a live view of every lock and unlock event across the vehicle fleet, event statistics, and a security overview that shows every rejected attack with its rejection reason

LAYOUT

The dashboard is divided into three main sections

- **Fleet overview:** count of registered vehicles, active keys, registered users, and total events



- **Recent events:** table of latest entries from vehicleEntries in Firestore, with the timestamp, user, vehicle, action, result, and rejection reason if applicable. Successes and failures are colour coded to easily spot attacks

Recent Events

Time	User ID	Vehicle ID	Action	Result	Source	Failure Reason
2026-04-28 00:32:47	attacker	vehicle_1	UNKNOWN	FAILURE	VACU	payload expired or timestamp out of range
2026-04-28 00:32:47	attacker	vehicle_1	UNKNOWN	FAILURE	VACU	payload expired or timestamp out of range
2026-04-28 00:32:26	attacker	vehicle_1	UNKNOWN	FAILURE	VACU	payload expired or timestamp out of range
2026-04-28 00:31:18	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	UNLOCK	SUCCESS	VACU	
2026-04-28 00:31:14	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	LOCK	SUCCESS	VACU	
2026-04-28 00:26:45	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	LOCK	SUCCESS	VACU	
2026-04-28 00:26:39	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	UNLOCK	SUCCESS	VACU	
2026-04-28 00:20:04	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	LOCK	FAILURE	VACU	no public key registered
2026-04-28 00:20:02	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	UNLOCK	FAILURE	VACU	no public key registered
2026-04-28 00:06:20	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	UNLOCK	FAILURE	VACU	no public key registered
2026-04-28 00:06:20	8HZ6sdWmx2VDxZrz4FJB	vehicle_1	UNLOCK	FAILURE	VACU	no public key registered

- **Security overview:** bar chart of failure events over a time range set by user (1 day / 3 days / 1 week / 1 month). Each bar is broken down by failure reason, so a manager can see what attack took place.



Testing was carried out across all four V-Model levels, with each type of test corresponding to a development phase. The unit tests check individual modules in isolation. The integration tests check the interfaces between components. The system tests verify end-to-end behaviour on real hardware. The threat simulator validates the security checks against the defined attack catalogue.

UNIT TESTING

The unit tests were written for components where the logic could be checked without needing the cloud implemented, BLE running or hardware connected.

The Android side has two test modules:

- **SecurityManagerTest (JVM unit tests):** Cover the parts of Security Manager that don't need Android running. Tests check nonce generation correctly produces 32 character alphanumeric strings, repeated payload generations produce unique nonces, **parsePayload** correctly rejects payloads with invalid number of fields or an invalid timestamp.
- **SecurityManagerInstrumentedTest (Android instrumented tests):** Must run on a real device or emulator, because key signing requires Android Keystore to be available. These tests check that **buildPayload** produces a correct four-field payload, that the timestamp is a valid millisecond value, that the signature field is not empty, and that consecutive payload generations produce different nonces. A separate test signs a known string with **signEcdsa**, and checks the output is valid Base64, to confirm the Keystore signing path works.

For the Python VACU, the **payload_parser** module was unit tested for:

- Correct parsing of payloads
- Rejection of malformed payloads
- Rejection of unknown commands
- Rejection of timestamps outside validity window, including timestamps with future dates
- The nonce cache accepts a fresh nonce, rejects a nonce repeat within the time window, and accepts the same nonce again once it has left the nonce cache

INTEGRATION TESTING

Integration tests verify that pairs of components work together correctly.

App and Firebase: The user login was tested using valid credentials, invalid credentials, and a user who exists in Firebase Authentication but has no profile document in Firestore. On user's first sign in, the public key is registered correctly to the user's Firestore profile document, verified by reading the document in the Firebase Console. The manager operations such as assigning keys, revoking keys, adding user, were tested through the Manager App and their results were checked on the corresponding Firestore documents.

App and VACU: The BLE connection between the App and VACU was tested by starting the VACU, and attempting to connect from the Car Key App. The connection state shows correctly on the home screen. The unlock and lock buttons are disabled when no connection is running to the VACU. If a connection loss occurs, the connection recovers automatically as the VACU rescans for BLE connections. Signed payloads are received by the VACU and parsed correctly.

VACU and Firestore: The VACU's cloud calls for the active key, the user's public key and audit log writes were tested through three conditions:

1. Valid active key with a registered public key (success)
2. Revoked key (rejected at stage 3)
3. Active key with user who has not signed in on phone, and therefore has no public key registered (rejected at stage 4)

The TTL caches were verified by checking the Firestore read counts in the Firebase Console while the system was running.

VACU and CANoe: The CAN message generation was verified by triggering an unlock from the app and verifying the Vehicle.DoorLock system variable changes from 1 to 0 in the CANoe trace window, and the corresponding CAN frame is sent out on the bus.

SYSTEM TESTING

System tests were end-to-end scenarios performed with the full system stack running: phone app, VACU, Firebase, CANoe and the JLR test rig.

Scenario	Expected Result	Actual Result
Manager assigns key, driver signs in, presses unlock	Test rig wakes up, SUCCESS event on dashboard	Pass
Driver presses lock	Test rig shut down, SUCCESS event on dashboard	Pass
Manager revokes key, driver presses unlock	Command rejected at VACU stage 3, FAILURE event with "no active key"	Pass
Driver signs out, signs back in on same phone	The existing key in Keystore is reused	Pass
New user signs in for first time	Public key generated and registered on Firestore, visible in Firestore user document	Pass
BLE disconnects mid session, driver tries to unlock	App shows "VACU not connected", no command sent	Pass
App closed and reopened with user logged in	Biometric unlock screen showed, BLE reconnects and command works	Pass

The latency from unlock/lock button press on app to test rig response was tested repeatedly and came to under a second.

SECURITY TESTING

The security testing used the **threat_sim.py** module, which sends payloads through the same payload validation checks that is used for real BLE traffic. Each result is sent to the Firestore audit log and shown on the dashboard.

Attack	Expected Stage	Result
Replay with expired timestamp	Stage 1 (timestamp out of range)	Rejected
Replay with same nonce inside validity window	Stage 2 (nonce already seen)	Rejected
Tampered command with original signature	Stage 4 (signature mismatch)	Rejected
Malformed payload	Stage 1 (parse fail)	Rejected
Empty payload	Stage 1 (parse fail)	Rejected
Unknown command (start_engine)	Stage 1 (command not in allow list)	Rejected
Future-dated payload (+1 hour)	Stage 1 (timestamp out of range)	Rejected
Revoked key	Stage 3 (key not active)	Rejected

The replay within validity window and tamper attacks require a payload captured from a real phone, as the threat simulator cannot produce a valid ECDSA signature without the user's private key.

ACCEPTANCE TESTING

Acceptance was tested against the original functional requirements from Phase 1. Almost every functional requirement has a corresponding pass in the tests above. The two requirements not fully met are:

- BLE channel encryption (FR-08 / NFR-04): The BLE traffic is not encrypted at the transport layer in this prototype. It caused issues with reliable BLE connection for the prototype, and enabling BLE Secure Connections is identified as future work.
- Hardware-backed VACU trust: The VACU runs as a software only script on the PC without secure boot or hardware attestation. Any physical attack on the VACU host PC will allow bypassing the validation pipeline. This is an accepted limitation of the prototype.

FUTURE WORK / LIMITATIONS

Future work for this project includes enabling BLE pairing and bonding so that BLE traffic is encrypted, rather than relying on the payload signature alone for integrity. Implementing a push notification channel from the cloud to the VACU would remove the small delay between a manager revoking a key and the VACU stopping accepting it. Adding a check that confirms the VACU has not been tampered with before it accepts any commands would extend the protection past the current software only setup. Allowing a user to move their digital key from one phone to another would also be needed for production use, since the private key is currently tied to the device that generated it. The DCC Digital Key 3.0 Specification defines a process for this. (Car Connectivity Consortium, 2026). Finally, replacing ECDSA with a signature scheme designed to resist quantum attacks is worth investigating for long term deployment of the system.

REFLECTION

This project was structured around using the V-Model, with each component developed through requirements analysis, system design, module design, and development phases, that mapped directly to testing phases. This approach worked well for me at first but I had to make a change mid-project which was hard to fit into the methodology. I learned V-Model is most useful when the requirements are stable.

I originally was using HMAC-SHA256, with a per key shared secret stored in Firestore. This worked well but upon further thinking and work on an abstract for a security conference, I had to realise that storing signing material on the cloud is a glaring security issue and isn't good enough for a system like this. I migrated the system to ECDSA P-256 and used Android Keystore instead. This asymmetric approach was a lot safer as only a public key was stored in the cloud. Looking back, I really wish I had realised this oversight earlier as it caused major headaches to refactor the system. It's best to not make it work once and run with it, but to design the cryptographic methods carefully and focus on it first, rather than doing the surrounding system first.

In terms of skills, the project was challenging as it used a wide range: Android with Kotlin and Jetpack Compose, Python with bleak BLE, CANoe with CAPL code, Firebase and Firestore, and Streamlit for the dashboard. I found the Bluetooth to be challenging and I couldn't manage to get Bluetooth Secure Connections working, which would enforce strict pairing and bonding between the app and VACU. It was also hard to make the modules operate with each other cleanly, and I learned that integration is where the main focus should be, as unit tests don't accomplish much when compared to running as part of a whole system.

For a project like this in future, I would definitely focus on my cybersecurity implementation first before looking at the whole system, as that caused a major roadblock for me.

CONCLUSION

This project set out to create a secure smartphone based digital car key system for fleet and rental management, and demonstrate real CAN bus behaviour on a JLR test rig. The final system meets that goal end to end. A driver logs on an Android app, sends a signed command over BLE to a VACU, which validates the command with a four stage pipeline before sending it to CANoe to a rig through CAN. A manager interface manages keys, users, and vehicles, A Streamlit dashboard shows real time monitoring of events on the system.

A main security point is that no signing secret exists between the phone and cloud. Only the public key is stored on the cloud which is hardware backed by Android Keystore. This is a major security benefit as it stops attackers from producing valid payloads on anything other than a compromised mobile device. A four stage verification process is robust and prevents common security attacks such as replay, man in the middle and payload tampering.

DECLARATION OF USE OF AI

The development of this project made use of Claude code for guidance, reviewing code and debugging, and explaining technical concepts. The table below shows the main areas where AI was used.

Task	Prompt
Android Keystore ECDSA P-256 setup	Help me setup ECDSA P-256 key pair using Android Keystore. Private key is stored per user

Migrating from HMAC signing to asymmetric	Whats the best way to migrate using per key HMAC secrets in the cloud to asymmetric signing with android keystore
Python ECDSA signature verification	Have an X.509 public key encoded as base64, how do I verify ecdsa signature in python
Firestore quota limits	Im hitting the firestore quota when im running the demo, help me troubleshoot
Firestore Security Rules	Help write security rules for firestore for role based access for drivers and managers, drivers can only change their own public key
Unit tests	Look at my payload format and suggest some unit tests to verify structure
Streamlit dashboard time range	Help add a time selector for my security graph on Streamlit, for 1 day, 3 day, week, month,
Plotly barchart fix	Chart is sorting security events alphabetically rather than by date, help me fix
BLE duplicate UUID issue	Getting a duplicate UUID error from VACU when starting the BLE on Android
BLE reconnection	How would I make the BLE gatt loop reconnect automatically on connection loss?

REFERENCES

- Car Connectivity Consortium. (2026). Car Connectivity Consortium | The Future of Digital Vehicle Access. [online] Available at: <https://carconnectivity.org/>
- Xie, X., Jiang, K., Dai, R., Lu, J., Wang, L., Li, Q. and Yu, J. (2023). Access Your Tesla without Your Awareness: Compromising Keyless Entry System of Model 3. Proceedings 2023 Network and Distributed System Security Symposium. [online] doi:<https://doi.org/10.14722/ndss.2023.24082>.
- Alrabady, A.I. and Mahmud, S.M. (2005). Analysis of Attacks Against the Security of Keyless-Entry Systems for Vehicles and Suggestions for Improved Designs. *IEEE Transactions on Vehicular Technology*, 54(1), pp.41–50. doi:<https://doi.org/10.1109/tvt.2004.838829>.
- OWASP. (n.d.). OWASP MASVS - OWASP Mobile Application Security. [online] Available at: <https://mas.owasp.org/MASVS/>.
- OWASP (n.d.). OWASP MASTG - OWASP Mobile Application Security. [online] Available at: <https://mas.owasp.org/MASTG/>.
- Bluetooth Core Specification (2024). *Core Specification* | Bluetooth® Technology Website. [online] Available at: <https://www.bluetooth.com/specifications/specs/core-specification-6-0/>.
- National Institute of Standards and Technology (2023) FIPS 186-5, Digital Signature Standard (DSS) [online] Available at: <https://csrc.nist.gov/pubs/fips/186-5/final>
- National Institute of Standards and Technology (2023) FIPS 180-4, Secure Hash Standard (SHS) [online] Available at: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>
- Google (2026) Android Keystore system. [online] Available at: <https://developer.android.com/privacy-and-security/keystore>.
- Google (2026) Biometric. [online] Available at: <https://developer.android.com/jetpack/androidx/releases/biometric>.

- Google (2026) Jetpack Compose UI App Development Toolkit. [online] Android Developers. Available at: <https://developer.android.com/compose>.
- Google (2026). Documentation | Firebase. [online] Firebase. Available at: <https://firebase.google.com/docs>.
- Vector Informatik GmbH. (2023). CANoe Family. [online] Available at: <https://www.vector.com/int/en/products/products-a-z/software/canoe/>.
- Blidh, H. (2022). bleak. [online] GitHub. Available at: <https://github.com/hbldh/bleak>.
- Streamlit.io. (2025). Streamlit Docs. [online] Available at: <https://streamlit.io/docs>.
- Forsberg, K. and Mooz, H. (1991). The Relationship of System Engineering to the Project Cycle. INCOSE International Symposium, 1(1), pp.57–65. doi:<https://doi.org/10.1002/j.2334-5837.1991.tb01484.x>.
- Google (2026). Guide to app architecture. [online] Android Developers. Available at: <https://developer.android.com/topic/architecture>.